Bongkey Eyewear MVP — Backend Engineering Architecture & Code Flow

Tech Stack: Go (Golang 1.23+), Gin Web Framework, PostgreSQL

Architecture: Layered Clean / Hexagonal (SRP Compliant)

Domain: Eyewear E-commerce (Frames, Sunglasses, Lenses, Guest Checkout)

Author/Role: Antigravity Backend Engineering System Guide

1. System Architecture & High-Level Design

Bongkey ব্যাকএন্ডটি Go এবং Gin ফ্রেমওয়ার্ক দিয়ে তৈরি একটি Production-grade, Single Responsibility Principle (SRP) মেনে চলা ক্লিন আর্কিটেকচার। এখানে কোনো ক্যাচ-অল (utils, helpers) প্যাকেজ নেই। প্রতিটি লেয়ার নিজের সীমানার মধ্যে দায়িত্ব পালন করে।



Core Layer Responsibilities & Strict Boundaries

- **Transport Layer (Handlers & Middleware):** HTTP request parse করে (JSON binding, URL params, headers), `context.Context` পাস করে সার্ভিস কল করে, এবং ডোমেইন রেজাল্ট বা এররকে HTTP Status Code ও JSON রেসপন্সে ম্যাপ করে। হ্যান্ডলারে *কোনো SQL কোয়ারি বা বিজনেস রুল থাকবে না।*
- **Service Layer:** বিজনেস লজিক, ট্রানজ্যাকশন অর্কেস্ট্রেশন, এবং ভ্যালিডেশনের মালিক। সার্ভিসের ফাংশনগুলো Gin Context (`*gin.Context`) বা HTTP সম্পর্কিত কোনো বিষয়ের ওপর নির্ভর করে না।

- **Repository Layer:** ডেটাবেস অপারেশন চালায়। Context-aware PostgreSQL কোয়ারি এক্সিকিউট করে। হ্যান্ডলারের রেসপন্স বা আংশিক ট্রানজ্যাকশন কমিট এখানে হয় না।
- **Security Boundary (`internal/security`):** ক্রিপ্টোগ্রাফিক অ্যালগরিদম (HMAC-SHA256, JWT, Constant-time compare) আইসোলেটেড রাখে। অন্য কোনো লেয়ার সরাসরি কাঁচা ক্রিপ্টো মেকানিজম লেখে না।

2. Complete File-by-File & Function-by-Function Reference

2.1 Server Entrypoint & Bootstrapping

`backend/cmd/server/main.go`

- `main()` : অ্যাপ্লিকেশনের এন্ট্রি পয়েন্ট। প্রথমে `config.Load()` কল করে কনফিগারেশন ভ্যালিডেট করে। কনফিগ ভ্যালিড হলে `serve(cfg)` কল করে সার্ভার শুরু করে। যেকোনো ফ্যাটাল এররে `log.Fatalf` দিয়ে টার্মিনেট করে।

`backend/cmd/server/server.go`

- `serve(cfg config.Config) error` :
  1. `logEnvironment(cfg)` দিয়ে `.env` সোর্স লগ করে।
  2. `database.Connect(cfg.DatabaseURL)` দিয়ে PostgreSQL কানেকশন তৈরি ও পিং টেস্ট করে। কানেকশন ফেইল হলে সিক্রেট ক্রপ করে সেফ এরর রিটার্ন করে।
  3. `database.MigrateAndSeed(cfg)` এক্সিকিউট করে স্কিমা মাইগ্রেশন ও অ্যাডমিন/প্রোডাক্ট সিডিং নিশ্চিত করে।
  4. `routes.SetupRouter(cfg).Run(":" + cfg.ServerPort)` চালিয়ে HTTP ট্রাফিক লিসেন করা শুরু করে (ব্লকিং কল)।
- `logEnvironment(cfg config.Config)` : কোন `.env` ফাইল থেকে ভ্যারিয়েবল লোড হয়েছে তা প্রিন্ট করে, কিন্তু সিক্রেট ভ্যালু কনসোলে প্রকাশ করে না।

`backend/internal/config/config.go`

- `Config` struct: সার্ভারের যাবতীয় এনভায়রনমেন্ট ভ্যারিয়েবল ধারণ করে (যেমন: `AppEnv`, `ServerPort`, `DatabaseURL`, `AdminEmail`, `AdminPassword`, `JWTSecret`, `GmailFrom`, `GmailAppPassword`, `ProjectRoot`)।
- `Load() (Config, error)` : রুট ডিরেক্টরি ডিটেক্ট করে, `.env` ফাইল স্ক্যান করে পরিবেশক ভ্যারিয়েবল সেট করে, এবং `validateSecurity()` চালায়।
- `(cfg Config) validateSecurity() error` : `JWTSecret` অন্তত ৩২ ক্যারেক্টার কিনা এবং কোনো ডিফল্ট টেমপ্লেট ভ্যালু রয়ে গেছে কিনা তা চেক করে। এছাড়াও Gmail ইউজার এবং অ্যাপ পাসওয়ার্ড একসাথে দেওয়া হয়েছে কিনা তা ভ্যালিডেট করে।
- `getEnv(key, fallback string) string` : ওএস এনভায়রনমেন্ট পড়ে, না পেলে ফলব্যাক ভ্যালু ব্যবহার করে।
- `findProjectRoot() (string, error)` : প্যারেন্ট ফোল্ডার ট্রাভার্স করে `.env.example` ও `docker-compose.yml` যেখানে আছে সেই রুট পাথ খুঁজে বের করে।
- `loadDotEnv(path string) (string, error)` : হ্যান্ড-ক্রাফটেড বাফার স্ক্যানার যা `.env` ফাইল নিরাপদে পার্স করে `os.Setenv` করে।
- `fileExists(path string) bool` : ফাইলটি বিদ্যমান ও নন-ডিরেক্টরি কিনা নিশ্চিত করে।

2.2 Database, Migrations & Seeding

`backend/internal/database/database.go`

- `DB *sql.DB` : গ্লোবাল প্যাকেজ-লেভেল PostgreSQL কানেকশন পুল।
- `Connect(databaseURL string) error` : `sql.Open("postgres", ...)` এবং `db.Ping()` এক্সিকিউট করে পুল রেডি করে।
- `Ready() bool` : ডিবি পুল ইনিশিয়ালাইজড কিনা বুলিয়ান রিটার্ন করে।
- `MigrateAndSeed(cfg config.Config) error` :
  1. `schemaSQL` এক্সিকিউট করে মূল টেবিলগুলো তৈরি করে (`admins`, `categories`, `products`, `orders`, `order_items`, `invoices`)।
  2. মাইগ্রেশন ফাইলগুলো ক্রমান্বয়ে এক্সিকিউট করে (`002_guest_order_access.sql`, `003_stock_quantity_nonnegative.sql`, `004_email_otp.sql`, `005_checkout_idempotency.sql`)।
  3. `seedAdmin(...)`, `seedCategories()`, এবং `seedProducts()` চালায়।
- `seedAdmin(db *sql.DB, email, password string) error` : ট্রানজ্যাকশন শুরু করে এবং `LOCK TABLE admins IN EXCLUSIVE MODE` নেয় যাতে একাধিক রেপ্লিকা একসাথে বুট করলেও ডুপ্লিকেট ওনার তৈরি না হয়। যদি কোনো অ্যাডমিন না থাকে, তবে bcrypt দিয়ে পাসওয়ার্ড হ্যাশ করে ওনার অ্যাকাউন্ট ইনসার্ট করে।
- `seedCategories() error` : ডিফল্ট ক্যাটাগরিগুলো ইনসার্ট করে (`ON CONFLICT (slug) DO NOTHING`)।

- `seedProducts() error` : ডেমো ফ্রেম ও সানগ্লাস প্রোডাক্টগুলো সিড করে তাদের টেকনিক্যাল মেজারমেন্ট ও স্টক কোয়ান্টিটিসহ।
- `slugify(value string) string` : টেক্সটকে লোয়ারকেস করে স্পেস ও স্পেশাল ক্যারেক্টার বাদ দিয়ে URL-বান্ধব স্লাগ তৈরি করে।

2.3 Security Subsystem (`backend/internal/security/`)

`backend/internal/security/admin_token.go`

- `IssueAdminToken(secret string, id int64) (string, error)` : অ্যাডমিনের জন্য HS256 স্বাক্ষরিত ২৪ ঘণ্টার মেয়াদের JWT সেশন টোকেন ইস্যু করে। এতে Issuer: `bongkey-api` , Audience: `bongkey-admin` , Subject: `id` , Role: `owner` যুক্ত থাকে।
- `VerifyAdminToken(secret, raw string) (int64, error)` : JWT পার্স ও সিগনেচার যাচাই করে। মেয়াদের সময়, অ্যালগরিদম (HS256), অডিয়েন্স এবং রোল (`owner`) ভ্যালিডেট করে অ্যাডমিন আইডি রিটার্ন করে।

`backend/internal/security/order_token.go`

- `NewOrderToken() (string, string, error)` : গেস্ট অর্ডারের জন্য ক্রিপ্টোগ্রাফিক র‍্যান্ডম ৩২-বাইট থেকে RawURLEncoding টোকেন তৈরি করে এবং এর SHA-256 হেক্স ডাইজেস্ট (স্টোরেজ হ্যাশ) রিটার্ন করে। ডেটাবেসে শুধু হ্যাশ সংরক্ষিত থাকে, কাঁচা টোকেন নয়।
- `IdempotentOrderToken(secret, idempotencyKey string) (string, string, error)` : চেকআউট রিট্রাই হলে ক্লায়েন্টকে একই গেস্ট টোকেন ফেরত দেওয়ার জন্য Idempotency Key দিয়ে HMAC-SHA256 ক্যালকুলেট করে ডিটারমিনিস্টিক টোকেন ও তার হ্যাশ তৈরি করে।
- `MatchesOrderToken(token, storedHash string) bool` : ইনপুট টোকেনটির হ্যাশ বের করে `subtle.ConstantTimeCompare` দিয়ে ডেটাবেস হ্যাশের সাথে তুলনা করে। এটি টাইমিং অ্যাটাক প্রতিরোধ করে।
- `orderTokenHash(token string) string` : টোকেনের SHA-256 হেক্স ডাইজেস্ট রিটার্ন করে।

`backend/internal/security/email_proof.go`

- `NewEmailCode() (string, error)` : ক্রিপ্টোগ্রাফিক র‍্যান্ডম ব্যবহার করে ৬ ডিজিটের ওটিপি কোড (যেমন: `482910`) জেনারেট করে।
- `EmailCodeHash(secret, email, code string) string` : সিক্রেট কি ও ইমেইল সল্টসহ ওটিপির HMAC-SHA256 হ্যাশ তৈরি করে, যাতে ডিবি লিক হলেও ওটিপি রিভার্স করা না যায়।
- `EmailKey(secret, kind, value string) string` : রেট লিমিট টেবিলে ব্যবহারের জন্য ইমেইল ও আইপির প্রিফিক্সড HMAC হ্যাশ কি তৈরি করে।
- `IssueEmailProof(secret, email string) (string, error)` : ওটিপি ভেরিফাই হওয়ার পর ১০ মিনিটের মেয়াদের একটি JWT "Email Proof" টোকেন ইস্যু করে (Audience: `bongkey-verified-email`)।
- `VerifyEmailProof(secret, raw string) (string, error)` : Email Proof টোকেন ভ্যালিডেট করে ভেরিফায়েড ইমেইল অ্যাড্রেস এক্সট্র্যাক্ট করে।

2.4 Middleware (`backend/internal/middleware/`)

`backend/internal/middleware/cors.go`

- `CORS() gin.HandlerFunc` : ক্রসব্রাউজার কল নিশ্চিত করতে `Access-Control-Allow-Origin: *` , মেথড এবং প্রয়োজনীয় হেডার (`Idempotency-Key` , `Authorization`) সেট করে। HTTP `OPTIONS` প্রি-ফ্লাইট রিকোয়েস্টে `204 No Content` দিয়ে রিটার্ন করে।

`backend/internal/middleware/admin_auth.go`

- `AdminAuth(service auth.Service) gin.HandlerFunc` : প্রোটেক্টেড রুটের গার্ড। `Authorization: Bearer <token>` এক্সট্র্যাক্ট করে `service.Authenticate(ctx, token)` কল করে। টোকেন ভুল বা মেয়াদোত্তীর্ণ হলে `401 Unauthorized` দিয়ে `c.Abort()` করে। ভ্যালিড হলে জিন কন্টেক্সটে `c.Set("admin_id", admin.ID)` সেট করে পরবর্তী হ্যান্ডলারের কাছে কন্ট্রোল পাঠায় (`c.Next()`)।

2.5 Auth & Admin Subsystem (`backend/internal/auth/`)

`backend/internal/auth/admin.go`

- `AdminRepository` interface: `ByEmail(ctx, email)` এবং `ByID(ctx, id)` মেথডের ডেফিনিশন।
- `PostgresRepository` struct: PostgreSQL থেকে সরাসরি অ্যাডমিন রেকর্ড রিড করে।
- `Service` struct: অ্যাডমিন লগইন ও অথেন্টিকেশন অর্কেস্ট্রেট করে।
  - `Login(ctx, email, password) (Admin, string, error)` : ইমেইল দিয়ে অ্যাকাউন্ট খোঁজে, পাসওয়ার্ডের bcrypt হ্যাশ কম্পেয়ার করে এবং রোল `owner` কিনা দেখে। সব সঠিক হলে `security.IssueAdminToken` দিয়ে JWT সেশন রিটার্ন করে। ভুল ক্রেডেনশিয়াল হলে নিরাপদ জেনেরিক এরর `ErrInvalidCredentials` দেয়।
  - `Authenticate(ctx, token) (Admin, error)` : টোকেন ভেরিফাই করে অ্যাডমিন আইডি বের করে এবং ডেটাবেসে অ্যাকাউন্টটি এখনো সক্রিয় ও ওনার রোলে আছে কিনা পুনরায় রি-ভ্যালিডেট করে।

2.6 Catalog Subsystem (`backend/internal/catalog/`)

`backend/internal/catalog/categories.go` & **`category_repository.go`**

- `CategoryService` :
  - `List(ctx)` : সমস্ত ক্যাটাগরি লোড করে।
  - `Save(ctx, id, category)` : ক্যাটাগরি নাম বাধ্যতামূলক কিনা চেক করে, নাম থেকে স্লাগ জেনারেট করে এবং রিপোজিটরির `Save` কল করে।
  - `Delete(ctx, id)` : ক্যাটাগরি মুছে ফেলে।
- `PostgresCategoryRepository` : `SELECT` , `INSERT` , `UPDATE` , এবং `DELETE` কোয়ারিগুলো এক্সিকিউট করে।

`backend/internal/catalog/products.go` & **`product_repository.go`**

- `ProductInput` struct: ওনার এডিট করতে পারে এমন ফিল্ডগুলোর ডেটা ট্রান্সফার অবজেক্ট (DTO)। এতে `ExpectedStockQuantity *int` অন্তর্ভুক্ত।
- `ProductService` :
  - `List(ctx, featuredOnly, includeInactive)` : ফিল্টার অনুযায়ী প্রোডাক্ট তালিকা আনে।
  - `GetBySlug(ctx, slug)` : পাবলিশড সিঙ্গেল প্রোডাক্ট আনে।
  - `Save(ctx, id, input)` : ভ্যালিডেশন করে (Category > 0, Name & Price > 0, Stock >= 0)। আপডেট অপারেশনের জন্য `ExpectedStockQuantity` থাকা বাধ্যতামূলক। ডিফল্ট ভ্যালু (স্লাগ, ব্র্যান্ড টায়ার, ফ্রেম টাইপ) ফিল করে রিপোজিটরিতে সেভ করে।
  - `Delete(ctx, id)` / `SetActive(ctx, id, active)` : প্রোডাক্ট ডিলিট বা পাবলিকেশন স্ট্যাটাস টগল করে।
- `PostgresProductRepository` :
  - `Save(ctx, id, input)` : **Optimistic Concurrency Control (OCC)** ইমপ্লিমেন্ট করে। আপডেটের সময় SQL হয়: `WHERE id=$21 AND stock_quantity=$22` । যদি অ্যাডমিন ফর্ম ওপেন রাখার সময়ে ব্যাকগ্রাউন্ডে চেকআউট হয়ে স্টক পরিবর্তিত হয়ে যায়, তখন `RowsAffected == 0` হয় এবং এটি `ErrStaleStock` রিটার্ন করে! এতে অ্যাডমিন অসাবধানতাবশত লাইভ সেলসের স্টক মুছে ফেলতে পারে না।
  - `scanProduct(...)` : স্ক্যান লজিক সেন্ট্রালাইজ করে।

2.7 Email OTP Subsystem (`backend/internal/emailotp/`)

`backend/internal/emailotp/gmail.go`

- `GmailSender` struct: জিমেইলের মাধ্যমে ওটিপি প্রেরক।
- `Send(ctx, to, code) error` : সরাসরি `smtp.gmail.com:465` -এ TLS v1.2+ কানেকশন খোলে। `context.WithTimeout(10s)` এবং `context.AfterFunc` ব্যবহার করে যাতে ক্লায়েন্ট রিকোয়েস্ট ক্যানসেল হলে সাথে সাথে সকেট ক্লোজ হয়ে যায়। প্লেইন টেক্সট ওটিপি মেইল পাঠায়। এরর হলে কোনো ক্রেডেনশিয়াল বা ইমেইল লগ না করে নিরাপদ `gmailFailure(stage)` লগ করে।
- `validAddress(value string) bool` : হেডার ইঞ্জেকশন রোধ করতে CRLF চেক ও RFC ফরম্যাট যাচাই করে।

`backend/internal/emailotp/repository.go`

- `PostgresStore` : ওটিপি চ্যালেঞ্জ ও রেট লিমিট ট্র্যাকিংয়ের ট্রানজ্যাকশনাল স্টোর।
  - `Reserve(ctx, emailKey, ipKey, codeHash) error` : একটি অ্যাটমিক ট্রানজ্যাকশনে ১ দিনের পুরোনো লগ ক্লিনিং করে, ক্লায়েন্ট আইপির ঘন্টায় সর্বোচ্চ ১০টি রিকোয়েস্টের লিমিট চেক করে (`email_otp_ip_limits`), এবং একই ইমেইলে ঘন্টায় সর্বোচ্চ ৩টি এবং প্রতি ১ মিনিটে ১টির বেশি ওটিপি পাঠানো আটকে দেয় (`email_otp_challenges`)। লিমিট পার হলে `ErrLimited` রিটার্ন করে।
  - `Release(ctx, emailKey, codeHash) error` : জিমেইল সেন্ডিং ফেইল করলে ওটিপি হ্যাশ ক্লিয়ার করে কোড বাতিল করে দেয় কিন্তু অ্যাবিউজ কাউন্টার বজায় রাখে।
  - `Consume(ctx, emailKey, codeHash) (bool, error)` : ভেরিফিকেশনের সময় অ্যাটমিকভাবে `attempts = attempts + 1` বাড়ায় (সর্বোচ্চ ৫ বার চেষ্টা করা যাবে), কোড মিলে গেলে `consumed_at = NOW()` সেট করে কোডটি সিঙ্গেল-ইউজ নিশ্চিত করে।

`backend/internal/emailotp/service.go`

- `Service` : ওটিপি লাইফসাইকেল কন্ট্রোল করে।
  - `Request(ctx, rawEmail, remoteIP) error` : ইমেইল নরম্যালাইজ করে, ৬ ডিজিটের কোড বানায়, স্টোরে রিজার্ভ করে, এবং জিমেইলে পাঠায়। ব্যর্থ হলে রিলিজ করে।
  - `Verify(ctx, rawEmail, code) (string, error)` : কোড কনজিউম করে এবং ম্যাচ করলে ১০ মিনিটের `IssueEmailProof` টোকেন রিটার্ন করে।

2.8 Orders & Checkout Subsystem (`backend/internal/orders/`)

`backend/internal/orders/checkout.go` & `checkout_repository.go`

- `CheckoutService` : সম্পূর্ণ চেকআউট প্রসেস এবং বিজনেস ইনভ্যারিয়েন্ট নিয়ন্ত্রণ করে।
  - `Place(ctx, request CheckoutRequest) (CheckoutReceipt, error)` :
    1. **Input & Idempotency Key Validation:** `ValidIdempotencyKey` নিশ্চিত করে কি-টি ১৬ থেকে ১২৮ ক্যারেক্টারের আলফানিউমেরিক স্ট্রিং।
    2. **Deadlock Prevention via Sorted Locking:** কার্টের প্রোডাক্ট আইডিগুলোকে `sort.Ints(productIDs)` দিয়ে ক্রমানুসারে সাজায়, যাতে কনকারেন্ট অর্ডারে ভিন্ন ভিন্ন ক্রমানুসারে রো লক হয়ে ডেটাবেস ডেডলক না ঘটে।
    3. **Request Fingerprinting:** `checkoutRequestHash` দিয়ে ক্লায়েন্টের মূল পেলোডের SHA-256 হ্যাশ বের করে (যাতে মেয়াদোত্তীর্ণ টোকেন বাদ থাকে)।
    4. **Distributed Idempotency with Advisory Lock:** ডেটাবেস ট্রানজ্যাকশনে প্রবেশ করে `pg_advisory_xact_lock(hashtextextended(keyHash, 0))` নেয়। কনকারেন্ট ডুপ্লিকেট রিকোয়েস্ট থাকলে প্রথমটি প্রসেস হবে এবং দ্বিতীয়টি লক মুক্ত হওয়া পর্যন্ত অপেক্ষা করে পূর্বের কমিট করা রিসিট দেখতে পাবে!
    5. **Payload Conflict Detection:** যদি একই Idempotency Key দিয়ে পূর্বে অর্ডার হয়ে থাকে কিন্তু বর্তমান পেলোডের সাথে হ্যাশ না মিলে, তবে `ErrIdempotencyConflict` (409 Conflict) দেয়!
    6. **Email Proof Verification:** `security.VerifyEmailProof` দিয়ে ওটিপি প্রুফ যাচাই করে।
    7. **Atomic Stock Reservation:** `UPDATE products SET stock_quantity = stock_quantity - $2 ... WHERE stock_quantity >= $2` দিয়ে অ্যাটমিকভাবে স্টক কাটে। স্টক কম হলে ট্রানজ্যাকশন রোলব্যাক হয়ে `ErrUnavailable` ফেরত দেয়।
    8. **Financial Calculations & Overflow Guard:** ইন্টিজার ওভারফ্লো গার্ডসহ সাবটোটাল ও ৮০ টাকা ডেলিভারি ফি যোগ করে গ্র্যান্ড টোটাল বানায়।
    9. **Atomic Insert:** অর্ডার (`ORD-XXXXXX`), প্রতিটি কার্ট আইটেম (`order_items`), এবং ইনভয়েস (`INV-XXXXXX`) একটিমাত্র অ্যাটমিক ট্রানজ্যাকশনে সেভ করে।
    10. **Deterministic Guest Token:** `security.IdempotentOrderToken` দিয়ে রিট্রাইয়ের ক্ষেত্রেও একই `guest_access_token` জেনারেট করে রিটার্ন করে।
- `PostgresCheckoutStore` : `InTransaction` মেথডের মাধ্যমে সেফ বিগিন, ডেফার্ড রোলব্যাক এবং কমিট হ্যান্ডেল করে।

`backend/internal/orders/access.go` & `repository.go`

- `AccessService` : প্রাইভেট অর্ডার ভিউ করার অথরাইজেশন রুলস চালায়।
  - `Guest(ctx, id, token) (Summary, error)` : গেস্ট ইউজারের ক্ষেত্রে টোকেনটি ডেটাবেসের `guest_token_hash` -এর সাথে `MatchesOrderToken` দিয়ে ম্যাচ করে। যদি না মিলে, তবে টোকেনটি কোনো ভ্যালিড `EmailProof` কিনা যাচাই করে এবং অর্ডারের ভেরিফায়েড ইমেইলের সাথে মিলে কিনা দেখে। অনুমতি না থাকলে `ErrNotFound` দেয় (যাতে আইডি এনুমেরেশন বা অস্তিত্ব জানা না যায়)।
  - `Admin(ctx, id) (Summary, error)` : অ্যাডমিন মিডলওয়্যার পার হয়ে আসা রিকোয়েস্টে সরাসরি অর্ডার সামারি ফেরত দেয়।
- `PostgresReader` : `ListAdmin` (অ্যাডমিন তালিকার জন্য) এবং `Find` (সিঙ্গেল অর্ডারের জন্য) মেথড সরবরাহ করে।

2.9 Handlers (`backend/internal/handlers/`)

Handler File	Handler Type	Key Methods	Responsibility & Invariants
`health_handler.go`	Functions	`Health`	সার্ভার অ্যালকোহল ও রানটাইম হেলথ স্ট্যাটাস (`{"status":"ok"}`) রিটার্ন করে।
`admin_handler.go`	`AdminHandler`	`Login` , `Me`	ক্রেডেনশিয়াল বাইন্ড করে ও `auth.Service.Login` কল করে। রেসপন্সে `Cache-Control: no-store` নিশ্চিত করে।
`category_handler.go`	`CategoryHandler`	`ListCategories` , `CreateCategory` , `UpdateCategory` , `DeleteCategory`	পাবলিক ও ওনার ক্যাটাগরি রিকোয়েস্ট হ্যান্ডেল করে। বডি সাইজ ৬৪KB-তে বাউন্ড রাখে (`http.MaxBytesReader`)।
`product_handler.go`	`ProductHandler`	`ListProducts` , `ListFeaturedProducts` , `AdminListProducts` , `GetProduct` , `CreateProduct` , `UpdateProduct` , `DeleteProduct` , `PatchProductStatus`	ক্যাটালগ প্রোডাক্ট ডিসপ্লে ও ক্রুড (CRUD)। ওনার এডিটে `expected_stock_quantity` পাস করে স্টেল স্টক কনফ্লিক্ট হ্যান্ডেল করে।
`upload_handler.go`	`UploadHandler`	`ProductImage` , `TryOnMockup`	সর্বোচ্চ ৫MB ইমেজ আপলোড নেয়। ম্যাজিক বাইট দিয়ে MIME ডিটেকশন (JPG, PNG, WebP) করে। র‍্যান্ডম হেক্স দিয়ে ইউনিক ফাইলনেম বানিয়ে `backend/uploads/` ফোল্ডারে সেভ করে।
`email_otp_handler.go`	`EmailOTPHandler`	`Request` , `Verify`	বডি ২KB বাউন্ড। `RemoteAddr` থেকে পিয়ার আইপি এক্সট্র্যাক্ট করে রেট লিমিটিং এনফোর্স করে ওটিপি পাঠায় ও ভেরিফাই করে।
`order_handler.go`	`CheckoutHandler`	`CreateOrder`	`Idempotency-Key` হেডার ও কার্ট আইটেম পার্স করে `orders.CheckoutService.Place` কল করে। `guest_access_token` সহ রিসিট দেয়।
`order_access_handler.go`	`OrderAccessHandler`	`GetOrder` , `GetAdminOrder`	`Authorization: Bearer <token>` থেকে গেস্ট টোকেন বা ইমেইল প্রুফ নিয়ে প্রাইভেট অর্ডার ডাটা রিটার্ন করে।
`order_list_handler.go`	`OrderListHandler`	`ListOrders`	অ্যাডমিন ড্যাশবোর্ডের জন্য সাম্প্রতিক অর্ডারগুলোর তালিকা রিটার্ন করে।

2.10 Routing Registry (`backend/internal/routes/routes.go`)

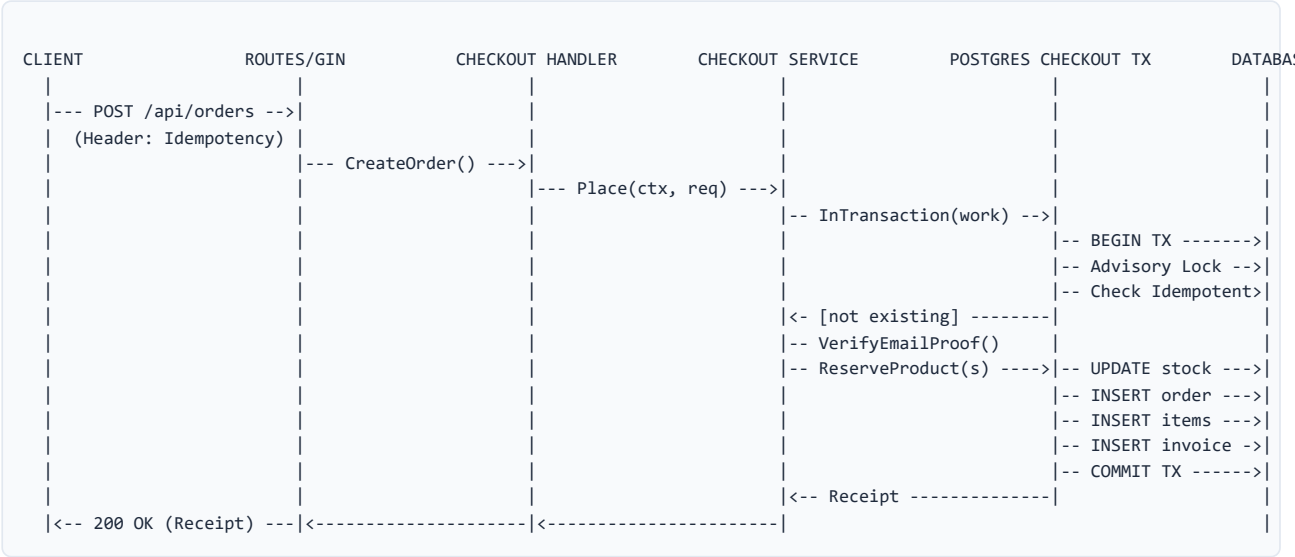
- `SetupRouter(cfg config.Config) *gin.Engine` :
  সমস্ত সার্ভিস ও ডিপেন্ডেন্সি ইনস্ট্যানশিয়েট করে এবং জিন ইঞ্জিনে রুট রেজিস্ট্রেশন করে:
  - গ্লোবাল মিডলওয়্যার: `middleware.CORS()`

- স্ট্যাটিক ফাইল সার্ভিং: `router.Static("/uploads", ...)`
- পাবলিক এন্ডপয়েন্টস: `GET /health`, `POST /api/admin/login`, `GET /api/products`, `GET /api/products/featured`, `GET /api/products/:slug`, `GET /api/categories`, `POST /api/orders`, `GET /api/orders/:id`, `POST /api/email-otp/request`, `POST /api/email-otp/verify`
- অ্যাডমিন গ্রুপ (`/api/admin`): `middleware.AdminAuth` মিডলওয়্যার দ্বারা সম্পূর্ণ সুরক্ষিত।

3. End-to-End Request & Function Call Flows

Flow A: Checkout & Order Placement (Cash on Delivery)

এটি ব্যাকএন্ডের সবচেয়ে গুরুত্বপূর্ণ ফ্লো, যাতে ট্রানজ্যাকশন, স্টক রিজার্ভেশন, এবং আইডেমপোটেন্সি জড়িত।



Detailed Hop-by-Hop Trace:

1. **Caller:** Frontend Checkout Form (`POST /api/orders`) with payload (Customer Name, Phone, Email, Address, Items) and `Idempotency-Key` header.
2. **Routing:** `routes.SetupRouter` matches `POST /api/orders` and dispatches to `handlers.CheckoutHandler.CreateOrder` .
3. **Transport Parsing:** `CreateOrder` verifies DB readiness, applies `http.MaxBytesReader(64KB)` , parses JSON, and validates `orders.ValidIdempotencyKey` . Then calls `orders.CheckoutService.Place` .
4. **Service Execution:**
   - Sorts product IDs with `sort.Ints` .
   - Computes `checkoutRequestHash` and `IdempotentOrderToken` .
   - Calls `s.Store.InTransaction` .
5. **Database Transaction & Locking:**
   - Acquires `pg_advisory_xact_lock` on idempotency hash.
   - Checks `FindIdempotentOrder` . If already completed with same payload, immediately returns existing receipt! If payload changed, returns `ErrIdempotencyConflict` (409).
   - Calls `security.VerifyEmailProof` to verify customer email.
   - Calls `ReserveProduct` for each product: `UPDATE products SET stock_quantity = stock_quantity - $2 ... WHERE stock_quantity >= $2` .
   - Calls `InsertOrder` (allocates `ORD-XXXXXX`), `InsertItem` , and `InsertInvoice` (allocates `INV-XXXXXX`).
   - Commits transaction.
6. **Response:** Handler formats JSON response containing order number, invoice number, totals, and `guest_access_token` with `200 OK` .
7. **Failure Paths:**
   - Missing required fields / invalid key -> `400 Bad Request`
   - Stock insufficient / product inactive -> `409 Conflict` (`item unavailable or out of stock`)
   - Reused idempotency key with different cart -> `409 Conflict`
   - Invalid or expired email OTP proof -> `400 Bad Request`

Flow B: Email OTP Request & Verification

```
1. REQUEST OTP:
   CLIENT -> POST /api/email-otp/request -> EmailOTPHandler.Request()
          -> Service.Request(email, peerIP)
          -> PostgresStore.Reserve(emailKey, ipKey, codeHash) [Checks IP <= 10/hr, Email <= 3/hr]
          -> GmailSender.Send(to, code) [TLS v1.2+, 10s Timeout]
          -> Returns 202 Accepted {"message": "verification code sent"}

2. VERIFY OTP:
   CLIENT -> POST /api/email-otp/verify -> EmailOTPHandler.Verify()
          -> Service.Verify(email, code)
          -> PostgresStore.Consume(emailKey, codeHash) [Atomic attempts+1, consumed_at=NOW()]
          -> security.IssueEmailProof(secret, email) [10 min JWT]
          -> Returns 200 OK {"email_proof": ""}
```

Detailed Hop-by-Hop Trace:

1. **Request Phase:**
   - Client submits `{"email": "user@example.com"}` to `/api/email-otp/request`.
   - `EmailOTPHandler.Request` extracts peer IP from `c.Request.RemoteAddr` (ignoring untrusted proxies) and bounds body to 2KB.
   - `emailotp.Service.Request` normalizes email, generates a 6-digit cryptographic code via `security.NewEmailCode()`, and hashes it using HMAC.
   - `PostgresStore.Reserve` runs an atomic PostgreSQL transaction updating `email_otp_ip_limits` and `email_otp_challenges`. If IP > 10 req/hr or Email > 3 req/hr or requested within 1 min cooldown, it returns `ErrLimited` (`429 Too Many Requests`).
   - `GmailSender.Send` connects to `smtp.gmail.com:465` over TLS, authenticates, and dispatches the plain text message. If SMTP fails, `PostgresStore.Release` invalidates the code challenge immediately.
2. **Verification Phase:**
   - Client submits `{"email": "...", "code": "123456"}` to `/api/email-otp/verify`.
   - `EmailOTPHandler.Verify` delegates to `Service.Verify`.
   - `PostgresStore.Consume` executes an atomic single-row update: increments attempts counter, and if code matches and is not expired and attempts < 5, sets `consumed_at = NOW()`.
   - Upon successful match, `security.IssueEmailProof` issues a signed, 10-minute JWT token with claim `bongkey-verified-email`. Client receives `{"email_proof": "..."}`.

Flow C: Private Order Lookup (Guest vs Verified Email vs Admin)

```
CLIENT REQUEST: GET /api/orders/:id
Header: Authorization: Bearer
          |
          v
routes.SetupRouter -> handlers.OrderAccessHandler.GetOrder()
          |
          v
orders.AccessService.Guest(ctx, orderID, token)
          |
          +--> Reader.Find(orderID) -> Loads Order + Invoice + GuestTokenHash + EmailVerified
          |
          +--> Branch 1: MatchesOrderToken(token, record.GuestTokenHash) == true?
          |               => Authorized! Return Summary.
          |
          +--> Branch 2: VerifyEmailProof(secret, token) == record.CustomerEmail && record.EmailVerified?
          |               => Authorized! Return Summary.
          |
          +--> Branch 3: Neither matched?
                          => Return ErrNotFound (404 Not Found). Safe, zero data leakage!
```

Flow D: Admin Authentication & Protected Route Access

1. **Admin Login (`POST /api/admin/login`):**
   - `handlers.AdminHandler.Login` parses `email` and `password`.
   - `auth.Service.Login` loads admin from `admins` table via `auth.PostgresRepository.ByEmail`.
   - Verifies role is `owner` and checks bcrypt password hash.
   - Issues 24-hour signed JWT session via `security.IssueAdminToken`.
   - Returns token with `Cache-Control: no-store`.
2. **Protected Admin Access (e.g., `GET /api/admin/orders`):**
   - Request hits `middleware.AdminAuth`.
   - Token is extracted from `Authorization: Bearer <token>`.
   - `security.VerifyAdminToken` checks signature, expiration, and claims.
   - `auth.PostgresRepository.ByID` checks that the owner account still exists in PostgreSQL.
   - Admin ID is set in context: `c.Set("admin_id", id)`.
   - Handler (e.g. `OrderListHandler.ListOrders`) is invoked and returns protected data.

4. Key Engineering Invariants & Architectural Best Practices

1. Deadlock Prevention in Multi-item Checkouts:
যদি ইউজার A কেনে [Product 2, Product 5] এবং একই সময়ে ইউজার B কেনে [Product 5, Product 2], তবে কনকারেন্ট ট্রানজ্যাকশনে লক নেওয়ার সময় ডেটাবেস ডেডলক ঘটতে পারত। বংকিতে `sort.Ints(productIDs)` দিয়ে আইডি ক্রমানুসারে লক নেওয়া হয়, যার ফলে ডেডলক গাণিতিকভাবে অসম্ভব।

2. Distributed Idempotency via PostgreSQL Advisory Locks:
একাধিক ব্যাকএন্ড রেপ্লিকা চললেও একই চেকআউট রিট্রাই ডুপ্লিকেট অর্ডার তৈরি করতে পারে না। `pg_advisory_xact_lock(hashtextextended(keyHash, 0))` ট্রানজ্যাকশন লেভেলে হ্যাশের ওপর ডিস্ট্রিবিউটেড লক বজায় রাখে।

3. Optimistic Concurrency Control (OCC) for Catalog Stock:
প্রোডাক্ট আপডেটে `UPDATE products ... WHERE id=$21 AND stock_quantity=$22` কুয়েরি ব্যবহার করা হয়। ওনার যখন কোনো প্রোডাক্টের ডিটেইলস এডিট করেন, সেই মুহূর্তের স্টক ফিল্ডে পাস হয়। ব্যাকগ্রাউন্ডে চেকআউট হয়ে স্টক পরিবর্তিত হলে `ErrStaleStock` রিটার্ন করে, যা ডেটা ওভাররাইট প্রতিহত করে।

4. Zero Information Leakage & Timing Attack Defense:
গেস্ট টোকেন যাচাইয়ে `subtle.ConstantTimeCompare` ব্যবহৃত হয়। প্রাইভেট অর্ডারের টোকেন ভুল হলে `403 Forbidden` না দিয়ে `404 Not Found` দেওয়া হয়, যাতে হ্যাকাররা আইডি এনুমারেট করতে না পারে। ডাটাবেস এররে কখনো ক্রেডেনশিয়াল বা কানেকশন স্ট্রিং লগ করা হয় না।